Ramakrishna Mission Vidyamandira

(An Autonomous College Under University of Calcutta)

Computer Science (Honors) Semester VI 2026

Paper: 6CMSMJC3 Practical

Submitted by
Class Roll Number: 340 Registration Number: B.Sc. 6th Semester Batch: 2023-27

INDEX

SI NO.	ASSIGNMNET STATEMENT	D-O-A	D-O-S	SIGNATURE
1.	Write an MPI program where each process prints its rank and the total number of processes in the communicator in the format: "Hello from process <rank> out of <size> processes".	16-01-26		
2.	Write an MPI program where process 0 sends a text message to all other processes. Every other process receives the message from process 0 and prints: "Process <rank> received: <message>".	16-01-26		
3.	Given an integer array of size N stored in process 0, write an MPI program to divide the array equally among all processes, each process computes the sum of its part, all partial sums are sent back to process 0 and process 0 computes and prints the total sum of the array.	16-01-26		
4.	Write an MPI program where process 0 creates an integer array of size N. Use MPI_Scatter to distribute equal chunks of the array to all processes. Each process multiplies every element of its chunk by 2. Use MPI_Gather to collect the modified chunks back into process 0. Process 0 prints the final modified array.	16-01-26		
5.	Write an MPI program where process 0 reads an integer value k, broadcast k to all processes using MPI_Bcast, each process computes $result = (rank + 1) \times k$, process 0 collects all	16-01-26		

	results and prints them.			
6.	Write an MPI program where each process generates M random integers, each process finds the maximum number in its local list. Use MPI_Reduce to find the global maximum among all processes. Process 0 prints the global maximum.	16-01-26		
7.	Write an MPI program to perform parallel search in an array where process 0 stores an integer array of size N and a search key 'key'. Distribute equal chunks of the array to all processes. Each process searches for key in its chunk. If any process finds the key, process 0 should print "Key found", otherwise print "Key not found".	16-01-26		
8.	Write an MPI program where all processes form a ring. Each process sends its rank value to the next process and receives a value from the previous process and then prints the value it received.	16-01-26		
9.	Write an MPI program to compute matrix-vector multiplication: $y = A \times x$, where process 0 stores matrix A of size N×N and vector x of size N, distribute rows of matrix A among processes, broadcast vector x to all processes, each process computes its partial result vector. Gather the partial results into process 0. Process 0 prints the final output vector y.	16-01-26		
10.	Write an MPI program to approximate the value of π using parallel computation. Use numerical integration method: $\pi \approx 4x\sum(1/(1+xi^2))\Delta x$(i=0 to N-1), where process 0 decides the total number of intervals	16-01-26		

	n, divide the computation among all processes. Each process computes its partial sum. Use MPI_Reduce to add all partial sums. Process 0 prints the estimated value of π.			
11.	Write an OpenMP program where a parallel region is created and each thread prints: "Hello from thread <thread_id> out of <total_threads> threads".	26-02-26		
12.	Write an OpenMP program to print numbers from 1 to N. Use #pragma omp parallel for to distribute iterations among threads. Each thread should print the number it is processing and its thread ID.	26-02-26		
13.	Write an OpenMP program to compute the sum of an integer array using #pragma omp parallel for. Don't use reduction initially. Observe incorrect results due to race conditions. Fix the problem using #pragma omp critical. Modify this program using reduction. Compare execution time with the critical version.	26-02-26		
14.	Write an OpenMP program to generate an array of random integers. Find the maximum element using reduction.	26-02-26		
15.	Write an OpenMP program to add two matrices of size N × N. Parallelize the outer loop.	26-02-26		
16.	Write an OpenMP program to multiply two N × N matrices. Use #pragma omp parallel for on outer loop. Compare performance for different thread counts.	26-02-26		
17.	Write an OpenMP program to approximate π using numerical integration. pi = 4	26-02-26		

	integration(0 to 1)(1/(1+x^2))dx Divide loop iterations among threads. Use reduction to compute the final result.			
18.	Write an OpenMP program that uses multiple threads. One thread produces numbers. Other threads consume numbers. Use synchronization constructs #pragma omp critical and #pragma omp atomic. Demonstrate proper synchronization to avoid data corruption.	26-02-26		
19.	Write an OpenMP program where a global variable counter is declared. Inside a parallel region, each thread modifies counter. First run without any clause and observe incorrect behaviour. Then use private(counter) and compare results. Explain the difference between shared and private variables.	26-02-26		
20.	Write an OpenMP program using #pragma omp sectionswhereone section computes factorial of a number, another section computes Fibonacci seriesand one more section prints prime numbers up to N. Each section runs in parallel.	26-02-26		
21.	Write an OpenMP program to compute the prefix sum of an array. Example: Input: [1,2,3,4], Output: [1,3,6,10].	26-02-26		
22.	Write an OpenMP program where an array of random numbers (0–9) is generated and compute frequency of each number in parallel.	26-02-26		
23.	Write an OpenMP program to sort an array using odd-even transposition sort. Parallelize comparison phases.	26-02-26		

24.	Write an OpenMP program to find all prime numbers up to N. Parallelize the checking loop. Use reduction to count total primes.	26-02-26		
25.	Write an OpenMP program where a parallel region contains another parallel region. Enable nested parallelism using omp_set_nested(1). Print thread IDs at both levels. Explain how nested parallelism behaves.	26-02-26		
26.	Write an OpenMP program using #pragma omp task to compute Fibonacci numbers recursively. Create tasks for recursive calls.	26-02-26		
27.	Write a CUDA program where each GPU thread prints "I am thread X in block Y".	07-03-26		
28.	Write a CUDA program to add two vectors A and B of size N.	07-03-26		
29.	Write a CUDA program where each thread multiplies one vector element by a constant scalar.	07-03-26		
30.	Write a CUDA kernel where each thread checks whether its assigned number is even or odd and store results in another array.	07-03-26		
31.	Write a CUDA program to reverse an array in parallel.	07-03-26		
32.	Write a CUDA program to search a key element in an array.	07-03-26		
33.	Write a CUDA program where each thread processes part of the array and find the maximum element using parallel reduction	07-03-26		
34.	Write a CUDA program to compute dot product of two vectors.	07-03-26		

35.	Given an array of numbers from 0–50, compute the frequency of each number using CUDA threads.	07-03-26		
36.	Write a CUDA program to add two matrices.	07-03-26		
37.	Write a CUDA program to multiply two matrices.	07-03-26		
38.	Write a CUDA program to compute matrix transpose.	07-03-26		
39.	Consider a grayscale image represented as a 2D array. Write a CUDA program where each thread increases pixel intensity by a constant value. Ensure pixel values remain within valid range.	07-03-26		
40.	Write a CUDA program where each thread processes a pixel. If the pixel value is greater than a threshold, then set it to 255, otherwise, set it to 0.	07-03-26		
41.	Write a CUDA program using 3D thread blocks to process volumetric data where each thread processes one voxel and applies a simple operation.	07-03-26		

MPI Programming Assignments

Q1. Write an MPI program where each process prints its rank and the total number of processes in the communicator in the format: "Hello from process <rank> out of <size> processes.

Source Code:

```
#include <stdio.h>
#include <mpi.h>
int main(int argc, char *argv[]) {
    int rank, size;
    // Initialize MPI
    MPI_Init(&argc, &argv);
    // Get process rank
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    // Get total number of processes
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    // Print message
    printf("Hello from process %d out of %d processes\n", rank, size);
    // Finalize MPI
    MPI_Finalize();
    return 0;
}
```

Output:

Case 1:

```
sukdeb@SUKDEB:~/sukdeb/HPC/MPI/Assignment$ mpicc 1.c -o 1
sukdeb@SUKDEB:~/sukdeb/HPC/MPI/Assignment$ mpirun -np 4 ./1
Hello from process 2 out of 4 processes
Hello from process 3 out of 4 processes
Hello from process 0 out of 4 processes
Hello from process 1 out of 4 processes
```

Case 2:

```
sukdeb@SUKDEB:~/sukdeb/HPC/MPI/Assignment$ mpicc 1.c -o 1
sukdeb@SUKDEB:~/sukdeb/HPC/MPI/Assignment$ mpirun -np 6 ./1
Hello from process 0 out of 6 processes
Hello from process 1 out of 6 processes
Hello from process 3 out of 6 processes
Hello from process 4 out of 6 processes
```

```
Hello from process 5 out of 6 processes
Hello from process 2 out of 6 processes
Hello from process 2 out of 6 processes
```

Q2. Write an MPI program where process 0 sends a text message to all other processes. Every other process receives the message from process 0 and prints: "Process <rank> received: <message>.

Source Code:

```
#include <stdio.h>
#include <string.h>
#include <mpi.h>
int main(int argc, char *argv[]) {
    int rank, size;
    char message[100];
    // Initialize MPI
    MPI_Init(&argc, &argv);
    // Get rank and size
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (rank == 0) {
        // Process 0 prepares message
        strcpy(message, "Hello from process 0");

        // Send to all other processes
        for (int i = 1; i < size; i++) {
                   MPI_Send(message,  strlen(message)  +  1,  MPI_CHAR,  i,  0,
MPI_COMM_WORLD);
        }
    }
    else {
        // Receive message from process 0
     MPI_Recv(message, 100, MPI_CHAR, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

        printf("Process %d received: %s\n", rank, message);
    }
    // Finalize MPI
    MPI_Finalize();
    return 0;
}
```

Output:

Case 1:

```
sukdeb@SUKDEB:~/sukdeb/HPC/MPI/Assignment$ mpicc 2.c -o 2
sukdeb@SUKDEB:~/sukdeb/HPC/MPI/Assignment$ mpirun -np 4 ./2
Process 1 received: Hello from process 0
Process 2 received: Hello from process 0
Process 3 received: Hello from process 0
```

Case 2:

```
sukdeb@SUKDEB:~/sukdeb/HPC/MPI/Assignment$ mpicc 2.c -o 2
sukdeb@SUKDEB:~/sukdeb/HPC/MPI/Assignment$ mpirun -np 6 ./2
Process 1 received: Hello from process 0
Process 2 received: Hello from process 0
Process 3 received: Hello from process 0
Process 4 received: Hello from process 0
Process 5 received: Hello from process 0
```

Q3. Given an integer array of size N stored in process 0, write an MPI program to divide the array equally among all processes, each process computes the sum of its part, all partial sums are sent back to process 0 and process 0 computes and prints the total sum of the array.

Source Code:

```
#include<stdio.h>
#include<mpi.h>
#include<time.h>
#include<stdlib.h>

int main(int argc, char *argv[]) {

    int rank, size;
    srand(time(NULL));

    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int n;

    //Input size (only root)
    if (rank == 0) {
        printf("Enter size of array:\n");
        scanf("%d", &n);
    }

    // Broadcast n to all processes
    MPI_Bcast(&n, 1, MPI_INT, 0, MPI_COMM_WORLD);

    int a[n];  // dynamic array

    //Input array (only root)
    if (rank == 0) {
        printf("Enter %d elements:\n", n);
        for (int i = 0; i < n; i++) {
            a[i]=rand() % 20; // random values between 0 and 9
            printf("%d ", a[i]);
        }
```

```
        printf("\n");
    }

    // Broadcast full array to all processes
    MPI_Bcast(a, n, MPI_INT, 0, MPI_COMM_WORLD);


    int chunk = n / size;
    int rem = n % size;

    int start, end;

    if (rank < rem) {
        start = rank * (chunk + 1);
        end = start + (chunk + 1);
    } else {
        start = rank * chunk + rem;
        end = start + chunk;
    }

    //Local computation
    int local_sum = 0;

    for (int i = start; i < end; i++) {
        local_sum += a[i];
    }

    printf("Process %d → Local sum = %d\n", rank, local_sum);

    //Reduce to get total sum
    int total = 0;

    MPI_Reduce(&local_sum, &total, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);

    //Final output
    if (rank == 0) {
        printf("\nTotal sum of array = %d\n", total);
    }
    MPI_Finalize();
    return 0;
}
```

Output:

Case 1:

```
sukdeb@SUKDEB:~/sukdeb/HPC/MPI/Assignment$ mpicc 3.c -o 3 && mpirun -np 6 ./3

Enter size of array:
14
Enter 14 elements:
15 17 7 16 11 17 15 9 9 1 1 18 10 18
Process 0 → Local sum = 39
Process 1 → Local sum = 44
Process 3 → Local sum = 10
Process 2 → Local sum = 24
Process 4 → Local sum = 19
Process 5 → Local sum = 28

Total sum of array = 164
```

Case 2:

```
sukdeb@SUKDEB:~/sukdeb/HPC/MPI/Assignment$ mpicc 3.c -o 3 && mpirun -np 6 ./3
Enter size of array:
7
Enter 7 elements:
3 9 17 13 12 17 6
Process 0 → Local sum = 12
Process 1 → Local sum = 17
Process 2 → Local sum = 13
Process 4 → Local sum = 17
Process 5 → Local sum = 6
Process 3 → Local sum = 12

Total sum of array = 77
```

Q4. Write an MPI program where process 0 creates an integer array of size N. Use MPI_Scatter to distribute equal chunks of the array to all processes. Each process multiplies every element of its chunk by 2. Use MPI_Gather to collect the modified chunks back into process 0. Process 0 prints the final modified array.

Source Code:

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {

    int rank, size;
    int N;
   // Full array (only in process 0)
    int *arr = NULL;
   // Local chunk
    int *local_arr;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    //Input (only process 0)
    if (rank == 0) {
        printf("Enter size of array (N):\n");
        scanf("%d", &N);

        arr = (int*)malloc(N * sizeof(int));

        printf("Enter %d elements:\n", N);
        for (int i = 0; i < N; i++) {
            scanf("%d", &arr[i]);
        }
        printf("\n");
    }

    //Send N to all processes
    MPI_Bcast(&N, 1, MPI_INT, 0, MPI_COMM_WORLD);

    //Calculate chunk size
    int chunk = N / size;
```

```
    int remainder = N % size;

    //Allocate memory for local chunk
    local_arr = (int*)malloc(chunk * sizeof(int));

    //Scatter data
    MPI_Scatter(arr, chunk, MPI_INT,
                local_arr, chunk, MPI_INT,
                0, MPI_COMM_WORLD);

    //Multiply each element by 2
    for (int i = 0; i < chunk; i++) {
        local_arr[i] *= 2;
    }

    //Gather data back to process 0
    MPI_Gather(local_arr, chunk, MPI_INT,
               arr, chunk, MPI_INT,
               0, MPI_COMM_WORLD);

    //Handle leftover elements (only process 0)
    if (rank == 0) {
        for (int i = chunk * size; i < N; i++) {
            arr[i] *= 2;
        }

        //Print final result
        printf("Modified array:\n");
        for (int i = 0; i < N; i++) {
            printf("%d ", arr[i]);
        }
        printf("\n");
    }

    MPI_Finalize();
    return 0;
}
```

Output:

Case 1:

```
sukdeb@SUKDEB:~/sukdeb/HPC/MPI/Assignment$ mpicc 4.c -o 4 && mpirun -np 4 ./4
Enter size of array (N):
7
Enter 7 elements:
1 2 3 4 5 6 7

Modified array:
2 4 6 8 10 12 14
```

Case 2:

```
sukdeb@SUKDEB:~/sukdeb/HPC/MPI/Assignment$ mpicc 4.c -o 4 && mpirun -np 6 ./4
Enter size of array (N):
5
Enter 5 elements:
32 12 45 64 23

Modified array:
64 24 90 128 46
```

Q5. Write an MPI program where process 0 reads an integer value k, broadcast k to all processes using MPI_Bcast, each process computes $result = (rank + 1) \times k$, process 0 collects all results and prints them.

Source Code:

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char *argv[]) {

    int rank, size;
   // input value
    int k;
   // local result
    int result;
    // fixed size array
    int all_results[100];

    //Initialize MPI
    MPI_Init(&argc, &argv);

    //Get rank and size
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    //Process 0 takes input
    if (rank == 0) {
        printf("Enter value of k:\n");
        scanf("%d", &k);
    }

    //Broadcast k to all processes
    MPI_Bcast(&k, 1, MPI_INT, 0, MPI_COMM_WORLD);

    //Each process computes result
    result = (rank + 1) * k;

    //Gather results in process 0
    MPI_Gather(&result, 1, MPI_INT,
               all_results, 1, MPI_INT,
               0, MPI_COMM_WORLD);
```

```
    //Process 0 prints results
    if (rank == 0) {
        printf("Results:\n");
        for (int i = 0; i < size; i++) {
            printf("%d ", all_results[i]);
        }
        printf("\n");
    }

    MPI_Finalize();

    return 0;
}
```

Output:

Case 1:

```
sukdeb@SUKDEB:~/sukdeb/HPC/MPI/Assignment$ mpicc 5.c -o 5 && mpirun -np 4 ./5
Enter value of k:
5
Results:
5 10 15 20
```

Case 2:

```
sukdeb@SUKDEB:~/sukdeb/HPC/MPI/Assignment$ mpicc 5.c -o 5 && mpirun -np 6 ./5
Enter value of k:
7
Results:
7 14 21 28 35 42
```

Q6. Write an MPI program where each process generates M random integers, each process finds the maximum number in its local list. Use MPI_Reduce to find the global maximum among all processes. Process 0 prints the global maximum.

Source Code:

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(int argc, char *argv[]) {
    int rank, size;
    int M;
    int local_max = -1;
    int global_max;
   // local array fixed size
    int arr[100];
    //Initialize MPI
    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    //Take input (only process 0)
    if (rank == 0) {
        printf("Enter number of random values:\n");
        scanf("%d", &M);
    }
    //Broadcast M
    MPI_Bcast(&M, 1, MPI_INT, 0, MPI_COMM_WORLD);
    //Generate random numbers
    srand(time(NULL) + rank); // different seed for each process
    printf("Process %d numbers: ", rank);
    for (int i = 0; i < M; i++) {
        arr[i] = rand() % 100; // random numbers (0–99)
        printf("%d ", arr[i]);
        // find local max
        if (arr[i] > local_max) {
            local_max = arr[i];
        }
    }
    printf("\n");
```

```
    //Reduce to find global max
    MPI_Reduce(&local_max, &global_max, 1, MPI_INT,
               MPI_MAX, 0, MPI_COMM_WORLD);

    //Process 0 prints result
    if (rank == 0) {
        printf("Global Maximum = %d\n", global_max);
    }
    //Finalize MPI
    MPI_Finalize();

    return 0;
}
```

Output:

Case 1:

```
Enter number of random values:
6
Process 0 numbers: 65 64 7 26 38 37
Process 2 numbers: 57 12 21 94 38 75
Process 3 numbers: 12 71 97 69 17 48
Process 1 numbers: 57 38 96 69 70 65
Global Maximum = 97
```

Case 2:

```
sukdeb@SUKDEB:~/sukdeb/HPC/MPI/Assignment$ mpicc 6.c -o 6 && mpirun -np 6 ./6
Enter number of random values:
5
Process 0 numbers: 79 75 22 59 43
Process 1 numbers: 21 46 22 60 3
Process 2 numbers: 18 61 45 44 79
Process 3 numbers: 6 47 25 89 71
Process 4 numbers: 95 33 47 2 75
Process 5 numbers: 75 77 81 15 12
Global Maximum = 95
```

Q7. Write an MPI program to perform parallel search in an array where process 0 stores an integer array of size N and a search key 'key'. Distribute equal chunks of the array to all processes. Each process searches for key in its chunk. If any process finds the key, process 0 should print "Key found", otherwise print "Key not found".

Source Code:

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char *argv[]) {

    int rank, size;
    int N, key;
    // full array
    int arr[100];
    int local_arr[100];

    int found = 0;
    // global result
    int result;
    //Initialize MPI
    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    //Input (only process 0)
    if (rank == 0) {
        printf("Enter size of array:\n");
        scanf("%d", &N);

        printf("Enter elements:\n");
        for (int i = 0; i < N; i++) {
            scanf("%d", &arr[i]);
        }
        printf("Enter key to search:\n");
        scanf("%d", &key);
    }

    //Broadcast N and key
    MPI_Bcast(&N, 1, MPI_INT, 0, MPI_COMM_WORLD);
```

```
    MPI_Bcast(&key, 1, MPI_INT, 0, MPI_COMM_WORLD);

    //Calculate chunk size
    int chunk = N / size;

    //Scatter array
    MPI_Scatter(arr, chunk, MPI_INT,
                local_arr, chunk, MPI_INT,
                0, MPI_COMM_WORLD);

    //Each process searches
    for (int i = 0; i < chunk; i++) {
        if (local_arr[i] == key) {
            found = 1;
        }
    }

    //Reduce (logical OR)
    MPI_Reduce(&found, &result, 1, MPI_INT,
               MPI_LOR, 0, MPI_COMM_WORLD);

    //Process 0 prints result
    if (rank == 0) {
        if (result == 1)
            printf("Key found\n");
        else
            printf("Key not found\n");
    }
    //Finalize MPI
    MPI_Finalize();
    return 0;
}
```

Output:

Case 1:

```
sukdeb@SUKDEB:~/sukdeb/HPC/MPI/Assignment$ mpicc 7.c -o 7 && mpirun -np 4 ./7
Enter size of array:
6
Enter elements:
2 3 5 1 6 8
```

```
Enter key to search:
5
Key found
```

Case 2:

```
sukdeb@SUKDEB:~/sukdeb/HPC/MPI/Assignment$ mpicc 7.c -o 7 && mpirun -np 4 ./7
Enter size of array:
7
Enter elements:
7 3 5 1 8 4 6
Enter key to search:
9
Key not found
```

Q8. Write an MPI program where all processes form a ring. Each process sends its rank value to the next process and receives a value from the previous process and then prints the value it received.

Source Code:

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char *argv[]) {

    int rank, size;
    int send_data, recv_data;

    //Initialize MPI
    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    //Determine next and previous process
    int next = (rank + 1) % size;
    int prev = (rank - 1 + size) % size;

    //Data to send
    send_data = rank;

    //Send to next and receive from previous
    MPI_Sendrecv(&send_data, 1, MPI_INT, next, 0,
                 &recv_data, 1, MPI_INT, prev, 0,
                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);

    //Print received value
    printf("Process %d received %d\n", rank, recv_data);

    //Finalize MPI
    MPI_Finalize();

    return 0;
}
```

Output:

Case 1:

```
sukdeb@SUKDEB:~/sukdeb/HPC/MPI/Assignment$ mpicc 8.c -o 8 && mpirun -np 4 ./8
Process 1 received 0
Process 2 received 1
Process 3 received 2
Process 0 received 3
```

Case 2:

```
sukdeb@SUKDEB:~/sukdeb/HPC/MPI/Assignment$ mpicc 8.c -o 8 && mpirun -np 6 ./8
Process 0 received 5
Process 1 received 0
Process 2 received 1
Process 3 received 2
Process 4 received 3
Process 5 received 4
```

Q9. Write an MPI program to compute matrix-vector multiplication: $y = A \times x$, where process 0 stores matrix A of size N×N and vector x of size N, distribute rows of matrix A among processes, broadcast vector x to all processes, each process computes its partial result vector. Gather the partial results into process 0. Process 0 prints the final output vector y.

Source Code:

```
#include <stdio.h>
#include <mpi.h>
#define N 4

int main(int argc, char * argv[])
{
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    int A[N][N];
    int x[N];

    int chunk_size = N / size;

    int local_A[chunk_size][N];

    int partial_result[chunk_size];
    int result[N];

    if (rank == 0)
    {
        printf("The Matrix A:\n");
        for (int i = 0; i < N; i++)
        {
            for (int j = 0; j < N; j++)
            {
                A[i][j] = i + j;
                x[j] = j+1;
                printf("%d ", A[i][j]);
            }

            printf("\n");
```

```
        }
        printf("The Vector X:\n");

        for (int i = 0; i < N; i++)
        {
            printf("%d ", x[i]);
        }
        printf("\n");

    }

    MPI_Scatter(A, chunk_size*N, MPI_INT, local_A, chunk_size*N, MPI_INT, 0,
MPI_COMM_WORLD);

    printf("Process %d received Rows:\n", rank);
    for (int i = 0; i < chunk_size; i++)
    {
        for (int j = 0; j < N; j++)
        {
            printf("%d ", local_A[i][j]);
        }
        printf("\n");
    }


    MPI_Bcast(x, N, MPI_INT, 0, MPI_COMM_WORLD);

    for (int i = 0; i < chunk_size; i++)
    {
        partial_result[i] = 0;
        for (int j = 0; j < N; j++)
        {
            partial_result[i] = partial_result[i] + local_A[i][j]*x[j];

        }

    }

    MPI_Gather(partial_result, chunk_size, MPI_INT, result, chunk_size, MPI_INT,
   0, MPI_COMM_WORLD);
```

```
    if (rank == 0)
    {
        printf("Resultant Vector:\n");
        for (int i = 0; i < N; i++)
        {
            printf("%d ", result[i]);
        }
        printf("\n");

    }
    MPI_Finalize();
    return 0;
}
```

Output:

Case 1:

```
sukdeb@SUKDEB:~/sukdeb/HPC/MPI/Assignment$ mpicc 9a.c -o 9a && mpirun -np 4 ./9a
The Matrix A:
0 1 2 3
1 2 3 4
2 3 4 5
3 4 5 6
The Vector X:
1 2 3 4
Process 0 received Rows:
Process 1 received Rows:
1 2 3 4
0 1 2 3
Process 2 received Rows:
2 3 4 5
Process 3 received Rows:
3 4 5 6
Resultant Vector:
20 30 40 50
```

Case 2:

```
sukdeb@SUKDEB:~/sukdeb/HPC/MPI/Assignment$ mpicc 9a.c -o 9a && mpirun -np 4 ./9a
The Matrix A:
0 1 2 3 4
```

1 2 3 4 5
2 3 4 5 6
3 4 5 6 7
4 5 6 7 8
The Vector X:
1 2 3 4 5
Process 1 received Rows:
1 2 3 4 5
Process 0 received Rows:
0 1 2 3 4
Process 3 received Rows:
3 4 5 6 7
Process 2 received Rows:
2 3 4 5 6
Resultant Vector:
40 55 70 85 32768

Q10. Write an MPI program to approximate the value of π using parallel computation. Use numerical integration method: $\pi \approx 4x\sum(1/(1+xi^2))\Delta x$(i=0 to N-1), where process 0 decides the total number of intervals n, divide the computation among all processes. Each process computes its partial sum. Use MPI_Reduce to add all partial sums. Process 0 prints the estimated value of π.

Source Code:

```
#include <stdio.h>
#include <mpi.h>
#include <math.h>

int main(int argc,char *argv[]){
    int rank,size;
    double global = 0.0;
    double local = 0.0;
    int n;
    double d_n;
    int base,rem;

    MPI_Init(&argc,&argv);
    MPI_Comm_rank(MPI_COMM_WORLD,&rank);
    MPI_Comm_size(MPI_COMM_WORLD,&size);

    if(rank == 0){
        printf("enter intervals: \n");
        scanf("%d",&n);
        d_n = 1.0/n;
    }

    MPI_Bcast(&d_n,1,MPI_DOUBLE,0,MPI_COMM_WORLD);
    MPI_Bcast(&n,1,MPI_INT,0,MPI_COMM_WORLD);

    base = n/size;
    rem = n%size;

    int start = rank * base + (rank < rem ? rank : rem);;
    int end = start + base + (rank < rem ? 1 : 0);

    for(int i = start;i<end;i++){
        double x_i = (i+0)* d_n;
        local += 1.0/(1.0 + x_i * x_i);
```

```
        }
        local = local * d_n;
        MPI_Reduce(&local,&global,1,MPI_DOUBLE,MPI_SUM,0,MPI_COMM_WORLD);

        if(rank == 0){
            printf("Process 0 prints the estimated value of π : %f\n",4.0*global);
        }

        MPI_Finalize();
        return 0;
    }
```

Output:

Case 1:

```
sukdeb@SUKDEB:~/sukdeb/HPC/MPI/Assignment$ mpicc 10.c -o 10 && mpirun -np 4 ./10
enter intervals:
5
Process 0 prints the estimated value of π : 3.334926
```

Case 2:

```
sukdeb@SUKDEB:~/sukdeb/HPC/MPI/Assignment$ mpicc 10.c -o 10 && mpirun -np 4 ./10
enter intervals:
7
Process 0 prints the estimated value of π : 3.281048
```

OpenMP Programming Assignments

Q11. Write an OpenMP program where a parallel region is created and each thread prints: "Hello from thread out of threads".

Source Code:

```
#include <stdio.h>
#include <omp.h>

int main() {
    //number of threads
    int th;
    printf("Enter the number of threads:\n");
    scanf("%d", &th);

    // Set the number of threads for parallel
    omp_set_num_threads(th);

    // Start parallel
    #pragma omp parallel
    {
        printf("Hello from thread %d out of %d threads\n",
                omp_get_thread_num(),
                omp_get_num_threads());
    }
    return 0;
}
```

Output:

Case 1:

```
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ gcc 1.c -fopenmp -o 1
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ ./1
Enter the number of Threads:
4
Hello from thread 2 out of 4 threads
Hello from thread 3 out of 4 threads
Hello from thread 0 out of 4 threads
Hello from thread 1 out of 4 threads
```

Case 2:

```
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ ./1
Enter the no of Threads:
6
Hello from thread 5 out of 6 threads
Hello from thread 2 out of 6 threads
Hello from thread 3 out of 6 threads
Hello from thread 4 out of 6 threads
Hello from thread 0 out of 6 threads
Hello from thread 1 out of 6 threads
```

Q12. Write an OpenMP program to print numbers from 1 to N. Use #pragma omp parallel for to distribute iterations among threads. Each thread should print the number it is processing and its thread ID.

Source Code:

```
#include <stdio.h>
#include <omp.h>
int main(){
    printf("Enter the number: \n");
    int x;
    scanf("%d",&x);
   //start parallel
    #pragma omp parallel for
    for(int i = 1;i<=x;i++){
        printf("thread %d processing number %d\n",omp_get_thread_num(),i);
    }
    return 0;
}
```

Output:

Case 1:

```
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ gcc 2.c -fopenmp -o 2
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ ./2
Enter the number:
4
thread 2 processing number 3
thread 0 processing number 1
thread 1 processing number 2
thread 3 processing number 4
```

Case 2:

```
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ ./2
Enter the number:
6
thread 0 processing number 1
thread 5 processing number 6
thread 1 processing number 2
thread 4 processing number 5
thread 2 processing number 3
```

thread 3 processing number 4

Q13. Write an OpenMP program to compute the sum of an integer array using #pragma omp parallel for. Don't use reduction initially. Observe incorrect results due to race conditions. Fix the problem using #pragma omp critical. Modify this program using reduction. Compare execution time with the critical version.

Source Code:

```
#include<stdio.h>
#include<stdlib.h>
#include<omp.h>
#include<time.h>
int main(){
    srand(time(NULL));
    int r_sum = 0,sum=0;
    int n = 20;
    double start,end;
    int arr[n];
    for(int i=0;i<n;i++){
        arr[i] = rand()%100;
        r_sum+=arr[i];
    }
    printf("\nThe initial array:\n");
    for(int i=0;i<n;i++){
        printf("%d ",arr[i]);
    }
    printf("\n");
    printf("original sum of this array is:%d\n",r_sum);
    sum = 0;
      printf("\nsum of array using #pragma omp for...\nthat might cause race
condition");
    start = omp_get_wtime();
    #pragma omp parallel for
    for(int i=0;i<n;i++){
        sum+=arr[i];
    }
    end = omp_get_wtime();
    printf("\nsum of this array is:%d\n",sum);
    printf("execution time:%f\n",end-start);
    sum = 0;
    printf("\nsum of array using #pragma omp for and critical");
    start = omp_get_wtime();
```

```
    #pragma omp parallel for
    for(int i=0;i<n;i++){
        #pragma omp critical
        sum+=arr[i];
    }
    end = omp_get_wtime();
    printf("\nsum of this array is:%d\n",sum);
    printf("execution time:%f\n",end-start);
    sum = 0;
    printf("\nsum calculation using #pragma omp for reduction");
    start = omp_get_wtime();
    #pragma omp parallel for reduction(+:sum)
    for(int i=0;i<n;i++){
        sum+=arr[i];
    }
    end = omp_get_wtime();
    printf("\nsum of this array is:%d\n",sum);
    printf("execution time:%f\n",end-start);
}
```

Output:

Case 1:

```
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ gcc 3.c -fopenmp -o 3
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ ./3

The initial array:
97 11 47 50 69 94 98 62 12 50 97 90 94 60 68 52 89 27 20 43
The sum of this original array is: 1230

sum of array using #pragma omp for...
that might cause race condition
sum of this array is: 1076
execution time: 0.002840

sum of array using #pragma omp for and critical
sum of this array is: 1230
execution time: 0.001204

sum calculation using #pragma omp for reduction
sum of this array is: 1230
```

execution time: 0.001218

Case 2:

```
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ gcc 3.c -fopenmp -o 3
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ ./3

The initial array:
68 78 0 16 28 50 10 32 83 24 77 1 80 60 85 93 18 3 61 65
The sum of this original array is: 932

sum of array using #pragma omp for...
that might cause race condition
sum of this array is: 932
execution time: 0.007716

sum of array using #pragma omp for and critical
sum of this array is: 932
execution time: 0.008098

sum calculation using #pragma omp for reduction
sum of this array is: 932
execution time: 0.007539
```

Q14. Write an OpenMP program to generate an array of random integers. Find the maximum element using reduction.

Source Code:

```
#include<stdio.h>
#include<stdlib.h>
#include<omp.h>
#include<time.h>
int main(){
    srand(time(NULL));
    int arr[10];
    printf("The initial array is:\n");
    for(int i = 0;i<10;i++){
        arr[i] = rand()%100;
    }
    int max = 0;
    for(int i = 0;i<10;i++){
        printf("%d ",arr[i]);
    }
    printf("\n");
    #pragma omp parallel for reduction(max:max)
    for(int i = 0;i<10;i++){
        if (max < arr[i]){
            max = arr[i];
        }
    }
    printf("The maximum element using reduction.:%d \n",max);
}
```

Output:

Case 1:

```
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ gcc 4.c -fopenmp -o 4
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ ./4
The initial array is:
81 5 55 11 99 57 22 81 1 1
The maximum element using reduction: 99
```

Case 2:

```
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ gcc 4.c -fopenmp -o 4
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ ./4
The initial array is:
79 58 17 7 85 20 83 72 9 88
The maximum element using reduction: 88
```

Q15. Write an OpenMP program to add two matrices of size N × N. Parallelize the outer loop.

Source Code:

```
#include<stdio.h>
#include<stdlib.h>
#include<omp.h>
#include<time.h>
int main(){
    int size;
    srand(time(NULL));
    printf("Enter the size of the N x N matrix: \n");
    scanf("%d",&size);
    int A[size][size];
    int B[size][size];
    int res[size][size];
    for(int i = 0;i<size;i++){
        for(int j = 0;j<size;j++){
            A[i][j] = rand()%100;
            B[i][j] = rand()%100;
        }
    }
    //  matrix A
    printf("\nThis is Matrix A:\n");
    for(int i = 0;i<size;i++){
        for(int j = 0;j<size;j++){
            printf("%3d ",A[i][j]);
        }
        printf("\n");
    }
    //  matrix B
    printf("\nThis is Matrix B:\n");
    for(int i = 0;i<size;i++){
        for(int j = 0;j<size;j++){
            printf("%3d ",B[i][j]);
        }
        printf("\n");
    }
    #pragma omp parallel for
    for(int i = 0;i<size;i++){
        for(int j = 0;j<size;j++){
```

```
            res[i][j] = A[i][j] + B[i][j];
        }
    }
    printf("\nThis is result matrix:\n");
    for(int i = 0;i<size;i++){
        for(int j = 0;j<size;j++){
            printf("%3d ",res[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

Output:

Case 1:

```
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ gcc 5.c -fopenmp -o 5
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ ./5
Enter the size of the N x N matrix:
2

This is Matrix A:
 10  40
 56  14

This is Matrix B:
 76  38
 31  24

This is result matrix:
 86  78
 87  38
```

Case 2:

```
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ gcc 5.c -fopenmp -o 5
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ ./5
Enter the size of the N x N matrix:
3
```

This is Matrix A:

52	93	93
20	54	33
82	55	45

This is Matrix B:

1	56	18
54	28	2
36	4	60

This is result matrix:

53	149	111
74	82	35
118	59	105

Q16. Write an OpenMP program to multiply two N × N matrices. Use #pragma omp parallel for on outer loop. Compare performance for different thread counts.

Source Code:

```
#include<stdio.h>
#include<stdlib.h>
#include<time.h>
#include<omp.h>
int main(){
    int size;
    srand(time(NULL));
    printf("Enter the size of the N x N matrix: \n");
    scanf("%d",&size);
    int A[size][size];
    int B[size][size];
    int res[size][size];
    double start,end;
    int max_thread = 10;
    for(int t = 1;t<=max_thread;t++){
        omp_set_num_threads(t);
        for(int i = 0;i<size;i++){
            for(int j = 0;j<size;j++){
                A[i][j] = rand()%100;
                B[i][j] = rand()%100;
            }
        }
        //  matrix A
        printf("\nThis is Matrix A:\n");
        for(int i = 0;i<size;i++){
            for(int j = 0;j<size;j++){
                printf("%3d ",A[i][j]);
            }
            printf("\n");
        }
        //  matrix B
        printf("\nThis is Matrix B:\n");
        for(int i = 0;i<size;i++){
            for(int j = 0;j<size;j++){
                printf("%3d ",B[i][j]);
            }
            printf("\n");
```

```
        }
        start = omp_get_wtime();
        #pragma omp parallel for
        for(int i = 0;i<size;i++){
            for(int j = 0;j<size;j++){
                int sum = 0;
                for(int k =0;k<size;k++){
                    sum += A[i][k] * B[k][j];
                }
                res[i][j] = sum;
            }
        }
        end = omp_get_wtime();
        printf("\nThis is result matrix:\n");
        for(int i = 0;i<size;i++){
            for(int j = 0;j<size;j++){
                printf("%3d ",res[i][j]);
            }
            printf("\n");
        }
        printf("\nThreads used: %d\n",t);
        printf("Execution time: %f seconds\n", end - start);
    }
    return 0;
}
```

Output:

Case 1:

```
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ gcc 6.c -fopenmp -o 6
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ ./6
Enter the size of the N x N matrix:
2

This is Matrix A:
 89  24
 38  17
```

This is Matrix B:
57 20
65 93

This is result matrix:
6633 4012
3271 2341

Threads used: 1
Execution time: 0.000032 seconds

This is Matrix A:
29 85
3 82

This is Matrix B:
95 14
42 46

This is result matrix:
6325 4316
3729 3814

Threads used: 2
Execution time: 0.000377 seconds

This is Matrix A:
49 97
52 57

This is Matrix B:
75 31
93 16

This is result matrix:
12696 3071
9201 2524

Threads used: 3
Execution time: 0.000224 seconds

```
This is Matrix A:
 66    65
 30    45

This is Matrix B:
 23    52
 89    19

This is result matrix:
7303 4667
4695 2415

Threads used: 4
Execution time: 0.000894 seconds

This is Matrix A:
 98    91
 38    81

This is Matrix B:
 21    88
  8    67

This is result matrix:
2786 14721
1446 8771

Threads used: 5
Execution time: 0.000362 seconds

This is Matrix A:
 55    81
 61    57

This is Matrix B:
 19    59
 15    63

This is result matrix:
2260 8348
2014 7190
```

Threads used: 6
Execution time: 0.000699 seconds

This is Matrix A:
43 94
99 11

This is Matrix B:
54 95
51 17

This is result matrix:
7116 5683
5907 9592

Threads used: 7
Execution time: 0.001469 seconds

This is Matrix A:
26 21
69 27

This is Matrix B:
28 8
66 67

This is result matrix:
2114 1615
3714 2361

Threads used: 8
Execution time: 0.001117 seconds

This is Matrix A:
87 7
30 44

This is Matrix B:
70 77
88 86

```
This is result matrix:
6706 7301
5972 6094

Threads used: 9
Execution time: 0.000800 seconds

This is Matrix A:
 59    45
 45    36

This is Matrix B:
 78    21
  2    88

This is result matrix:
4692 5199
3582 4113

Threads used: 10
Execution time: 0.001129 seconds
```

Case 2:

```
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ gcc 6.c -fopenmp -o 6
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ ./6
Enter the size of the N x N matrix:
3

This is Matrix A:
 52    80    81
 10    48     5
 10    61    69

This is Matrix B:
 93    61    90
 55    90    62
 54     0    93

This is result matrix:
```

13610 10372 17173
3840 4930 4341
8011 6100 11099

Threads used: 1
Execution time: 0.000034 seconds

This is Matrix A:
6 14 85
13 25 67
12 39 84

This is Matrix B:
58 22 96
2 17 46
71 44 72

This is result matrix:
6411 4110 7340
5561 3659 7222
6738 4623 8994

Threads used: 2
Execution time: 0.000625 seconds

This is Matrix A:
34 27 37
96 91 0
2 22 80

This is Matrix B:
47 34 84
47 9 12
58 68 7

This is result matrix:
5013 3915 3439
8789 4083 9156
5768 5706 992

Threads used: 3

Execution time: 0.000496 seconds

This is Matrix A:

17	61	63
88	52	71
4	84	92

This is Matrix B:

93	42	81
27	80	37
58	84	73

This is result matrix:

6882 10886 8233
13706 13820 14235
7976 14616 10148

Threads used: 4
Execution time: 0.000265 seconds

This is Matrix A:

20	73	98
23	83	73
5	61	84

This is Matrix B:

89	11	25
53	45	15
90	18	76

This is result matrix:

14469 5269 9043
13016 5302 7368
11238 4312 7424

Threads used: 5
Execution time: 0.000236 seconds

This is Matrix A:

51	3	57
93	32	97

```
 54    66    81

This is Matrix B:
 25     4    74
 61    29    77
 18    27    16

This is result matrix:
2484 1830 4917
6023 3919 10898
6834 4317 10374

Threads used: 6
Execution time: 0.000295 seconds

This is Matrix A:
 52    21     2
  3    85    30
 40    47    37

This is Matrix B:
 57    87    95
 59    64    70
 81    95     4

This is result matrix:
4365 6058 6418
7616 8551 6355
8050 10003 7238

Threads used: 7
Execution time: 0.001129 seconds

This is Matrix A:
 69    65    63
 31    33    45
 17    23    37

This is Matrix B:
 82    54    15
 18    49    14
```

```
 50    39    78

This is result matrix:
9978 9368 6859
5386 5046 4437
3658 3488 3463

Threads used: 8
Execution time: 0.000835 seconds

This is Matrix A:
 34    37    56
 93    53    43
 44    25    19
This is Matrix B:
 40    71    19
 96    40    42
 65    62    40

This is result matrix:
8552 7366 4440
11603 11389 5713
5395 5302 2646
Threads used: 9
Execution time: 0.001559 seconds

This is Matrix A:
 77    58     3
 28     6    11
 30    36    69
This is Matrix B:
  2    62    56
 20    52    43
 45    19    92
This is result matrix:
1449 7847 7082
671 2257 2838
3885 5043 9576

Threads used: 10
Execution time: 0.001243 seconds
```

Q17. Write an OpenMP program to approximate π using numerical integration. pi = 4 integration(0 to 1)(1/(1+x^2))dx Divide loop iterations among threads. Use reduction to compute the final result.

Source Code:

```
#include<stdio.h>
#include<stdlib.h>
#include<omp.h>
#include<time.h>
int main(){
    int interval;
    double d_x,t_sum;
    printf("Enter the number of intervals: \n");
    scanf("%d",&interval);
    d_x = 1.0/interval;
    t_sum = 0;
    #pragma omp parallel for reduction(+:t_sum)
    for(int i = 0;i<interval;i++){
        double x_i = (0.5+i)* d_x;
        t_sum += 1.0/(1.0 + x_i * x_i);
    }
    printf("value of pi is %f \n",4*t_sum*d_x);
}
```

Output:

Case 1:

```
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ gcc 7.c -fopenmp -o 7
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ ./7
Enter the number of intervals:
4
value of pi is 3.146801
```

Case 2:

```
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ gcc 7.c -fopenmp -o 7
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ ./7
Enter the number of intervals:
7
value of pi is 3.143293
```

Q18. Write an OpenMP program that uses multiple threads. One thread produces numbers. Other threads consume numbers. Use synchronization constructs #pragma omp critical and #pragma omp atomic. Demonstrate proper synchronization to avoid data corruption.

Source Code:

```
#include<stdio.h>
#include<omp.h>
#include<time.h>
#include<stdlib.h>
#define N 5
int main(){
    int array[N];
    int count = 0;
    srand(time(NULL));
    #pragma omp parallel shared(array, count)
    {
        if(omp_get_thread_num() == 0){
            for(int i = 0;i<N;i++){
                #pragma omp critical
                {
                    if (count < N){
                        array[count] = rand()%100;
                        int value = array[count] ;
                        #pragma omp atomic
                        count++;
                        printf("producer produced %d (count = %d)\n", value, count);
                    }
                }
            }
        }
        else{
            for(int i = 0;i<N;i++){
                #pragma omp critical
                {
                    if (count > 0){
                    int value = array[count-1] ;
                    #pragma omp atomic
                    count--;
                    printf("consumer consumed %d (count = %d)\n", value, count);
                    }
```

```
                    }
                }
            }
        }
}
```

Output:

Case 1:

```
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ gcc 8.c -fopenmp -o 8
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ ./8
producer produced 83 (count = 1)
producer produced 7 (count = 2)
producer produced 20 (count = 3)
producer produced 29 (count = 4)
producer produced 11 (count = 5)
```

Case 2:

```
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ gcc 8.c -fopenmp -o 8
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ ./8
producer produced 45 (count = 1)
producer produced 47 (count = 2)
producer produced 87 (count = 3)
producer produced 87 (count = 4)
producer produced 84 (count = 5)
```

Q19. Write an OpenMP program where a global variable counter is declared. Inside a parallel region, each thread modifies counter. First run without any clause and observe incorrect behaviour. Then use private(counter) and compare results. Explain the difference between shared and private variables.

Source Code:

```
#include<stdio.h>
#include<stdlib.h>
#include<omp.h>
int count = 0;
int main(){
    // int count = 0;
    printf("case 1(without using any clause)\n");
    printf("at first count is %d\n",count);
    count+=1;
    printf("after increment count is %d\n",count);
    count = 0;
    printf("\n\ncase 2(using only palallel)\n");
    printf("at first count is %d\n",count);
    omp_set_num_threads(4);
    #pragma omp parallel
    {
        count+=1;
        printf("value of count inside the thread %d\n",count);
    }
    printf("after increment count is %d\n",count);
    count = 0;
    printf("\n\ncase 3(using palallel private(count)\n");
    printf("at first count is %d\n",count);
    omp_set_num_threads(4);
    #pragma omp parallel private(count)
    {
        count+=1;
        printf("value of count inside the thread %d\n",count);
    }
    printf("after increment count is %d\n",count);

}
```

Output:

Case 1:

```
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ gcc 9.c -fopenmp -o 9
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ ./9
case 1(without using any clause)
at first count is 0
after increment count is 1

case 2(using only palallel)
at first count is 0
value of count inside the thread 1
value of count inside the thread 1
value of count inside the thread 1
value of count inside the thread 2
after increment count is 2

case 3(using palallel private(count)
at first count is 0
value of count inside the thread 1
value of count inside the thread 1
value of count inside the thread 1
value of count inside the thread 30216
after increment count is 0
```

Case 2:

```
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ gcc 9.c -fopenmp -o 9
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ ./9
case 1(without using any clause)
at first count is 0
after increment count is 1

case 2(using only palallel)
at first count is 0
value of count inside the thread 1
value of count inside the thread 2
value of count inside the thread 3
value of count inside the thread 4
after increment count is 4
```

case 3(using palallel private(count)
at first count is 0
value of count inside the thread 1
value of count inside the thread 1
value of count inside the thread 1
value of count inside the thread 31006
after increment count is 0

Q20. Write an OpenMP program using #pragma omp sections where one section computes factorial of a number, another section computes Fibonacci series and one more section prints prime numbers up to N. Each section runs in parallel.

Source Code:

```
#include<stdio.h>
#include<omp.h>
int fact(int a){
    if(a==0||a == 1){
        return 1;
    }
    else if(a<0){
        return 0;
    }
    else{
        return a*fact(a-1);
    }
}
void fibbo(int n){
    int a = 0,b = 1,c;
    for(int i = 0;i<n;i++){
        printf("%d ",a);
        c = a+b;
        a = b;
        b = c;
    }
    printf("\n");
}
void prime(int n){
    for(int j = 2;j<=n;j++){
        int flag = 1;
        for(int i = 2;i*i<=j;i++){
            if(j%i==0){
                flag = 0;
                break;
            }
        }
        if(flag){
            printf("%d ",j);
        }
    }
```

```
    printf("\n");
}
int main(){
    int fac_num = 5;
    int prime_r = 59;
    int fibbo_r = 10;
    #pragma omp parallel sections
    {
        #pragma omp section
        {
            printf("The factorial of %d : %d\n",fac_num,fact(fac_num));
        }
        #pragma omp section
        {
            printf("The fibbonaci range upto %d :\n",fibbo_r);
            fibbo(fibbo_r);

        }
        #pragma omp section
        {
            printf("The prime range upto %d :\n",prime_r);
            prime(prime_r);
        }
    }
}
```

Output:

Case 1:

```
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ gcc 10.c -fopenmp -o 10
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ ./10
The fibbonaci range upto 10 :
0 1 1 2 3 5 8 13 21 34
The factorial of 6 : 720
The prime range upto 59 :
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59
```

Case 2:

```
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ gcc 10.c -fopenmp -o 10
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ ./10
```

The prime range upto 59 :
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59
The fibbonaci range upto 10 :
0 1 1 2 3 5 8 13 21 34
The factorial of 6 : 720

Q21. Write an OpenMP program to compute the prefix sum of an array. Example: Input: [1,2,3,4], Output: [1,3,6,10]

Source Code:

```
#include<stdio.h>
#include<omp.h>
#include<time.h>
#include<stdlib.h>
#define N 10
int main(){
    int arr[N];
    int arr_2[N];
    int sum = 0;
    srand(time(NULL));
    for(int i = 0;i<N;i++){
        arr[i] = rand()%10;
    }
    for(int i = 0;i<N;i++){
        printf("%d ",arr[i]);
    }
    printf("\n");
    #pragma omp parallel for reduction(+:sum)
    for(int i = 0;i<N;i++){
        sum = 0;
        for(int j = 0;j<=i;j++){
            sum+=arr[j];
        }
        arr_2[i] = sum;
    }
    for(int i = 0;i<N;i++){
        printf("The prefix sum of an array:%d ",arr_2[i]);
    }
}
```

Output:

Case 1:

```
b@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ gcc 11.c -fopenmp -o 11
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ ./11
0 1 1 9 5 8 5 6 8 0
The prefix sum of an array:0
```

```
The prefix sum of an array:1
The prefix sum of an array:2
The prefix sum of an array:11
The prefix sum of an array:16
The prefix sum of an array:24
The prefix sum of an array:29
The prefix sum of an array:35
The prefix sum of an array:43
The prefix sum of an array:43
```

Case 2:

```
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ gcc 11.c -fopenmp -o 11
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ ./11
2 5 3 8 5 6 7 8 7 0
The prefix sum of an array:2
The prefix sum of an array:7
The prefix sum of an array:10
The prefix sum of an array:18
The prefix sum of an array:23
The prefix sum of an array:29
The prefix sum of an array:36
The prefix sum of an array:44
The prefix sum of an array:51
The prefix sum of an array:51
```

Q22. Write an OpenMP program where an array of random numbers (0–9) is generated and compute frequency of each number in parallel.

Source Code:

```
#include<stdio.h>
#include<stdlib.h>
#include<omp.h>
#include<time.h>
int main(){
    srand(time(NULL));
    int n;
    printf("Enter the number of elements in the array:\n");
    scanf("%d",&n);
    int occr[10] = {0};
    int a[n];
    for(int i = 0;i<n;i++){
        a[i] = rand()%10;
    }
    printf("The unsorted array is\n");
    for(int i = 0;i<n;i++){
        printf("%2d ",a[i]);
    }
    printf("\n");
    #pragma omp parallel for
    for(int i = 0;i<n;i++){
        #pragma omp atomic
        occr[a[i]]++;
    }

    for(int i = 0;i<10;i++){
        if(occr[i]!=0){
            printf("The occurance of %d is %d\n",i,occr[i]);
        }
    }
}
```

Output:

Case 1:

```
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ gcc 12.c -fopenmp -o 12
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ ./12
Enter the number of elements in the array:
4
The unsorted array is
 0    6    6    8
The occurance of 0 is 1
The occurance of 6 is 2
The occurance of 8 is 1
```

Case 2:

```
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ gcc 12.c -fopenmp -o 12
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ ./12
Enter the number of elements in the array:
7
The unsorted array is
 8    9    0    0    2    5    1
The occurance of 0 is 2
The occurance of 1 is 1
The occurance of 2 is 1
The occurance of 5 is 1
The occurance of 8 is 1
The occurance of 9 is 1
```

Q23. Write an OpenMP program to sort an array using odd-even transposition sort. Parallelize comparison phases.

Source Code:

```
#include <stdio.h>
#include <omp.h>
#include<stdlib.h>
#include<time.h>
int main() {
    srand(time(NULL));
    int n;
    printf("Enter the number of elements in the array:\n");
    scanf("%d",&n);
    int a[n];
    for(int i = 0;i<n;i++){
        a[i] = rand()%100;
    }
    printf("the unsorted array is\n");
    for(int i = 0;i<n;i++){
        printf("%3d ",a[i]);
    }
    for (int phase = 0; phase < n; phase++) {
        if (phase % 2 == 0) {
            #pragma omp parallel for
            for (int i = 1; i < n; i += 2) {
                if (a[i-1] > a[i]) {
                    int temp = a[i];
                    a[i] = a[i-1];
                    a[i-1] = temp;
                }
            }
        } else {
            #pragma omp parallel for
            for (int i = 1; i < n-1; i += 2) {
                if (a[i] > a[i+1]) {
                    int temp = a[i];
                    a[i] = a[i+1];
                    a[i+1] = temp;
                }
            }
        }
```

```
    }
    printf("\nSorted array:\n");
    for (int i = 0; i < n; i++)
        printf("%3d ", a[i]);
    printf("\n");
    return 0;
}
```

Output:

Case 1:

```
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ gcc 13.c -fopenmp -o 13
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ ./13
Enter the number of elements in the array:
4
the unsorted array is
 29   80   38    9
Sorted array:
  9   29   38   80
```

Case 2:

```
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ gcc 13.c -fopenmp -o 13
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ ./13
Enter the number of elements in the array:
7
the unsorted array is
 40    9   51   92   13   76   19
Sorted array:
  9   13   19   40   51   76   92
```

Q24. Write an OpenMP program to find all prime numbers up to N. Parallelize the checking loop. Use reduction to count total primes.

Source Code:

```
#include<stdio.h>
#include<stdlib.h>
#include<omp.h>
int is_prime(int a){
    if(a>0){
        if (a == 1){
            return 0;
        }
        else if(a == 2){
            return 1;
        }
        else{
            for(int i = 2;i*i<=a;i++){
                if(a%i==0){
                    return 0;
                }
            }
            return 1;
        }
    }
}
int main(){
    int n,count = 0;
    printf("Enter the number to check prime upto: \n");
    scanf("%d",&n);
    #pragma omp parallel for reduction(+:count)
    for(int i = 1;i<n;i++){
        if(is_prime(i)){
            count++;
        }
    }
    printf("\nThe Total %d prime numbers upto %d\n",count,n);
}
```

Output:

Case 1:

```
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ gcc 14.c -fopenmp -o 14clear
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ ./14
Enter the number to check prime upto:
11

The Total 4 prime numbers upto 11
```

Case 2:

```
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ gcc 14.c -fopenmp -o 14clear
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ ./14
Enter the number to check prime upto:
32

The Total 11 prime numbers upto 32
```

Q25. Write an OpenMP program where a parallel region contains another parallel region. Enable nested parallelism using omp_set_nested(1). Print thread IDs at both levels. Explain how nested parallelism behaves.

Source Code:

```
#include<stdio.h>
#include<omp.h>
int main(){
    omp_set_num_threads(2);
    omp_set_nested(1);
    #pragma omp parallel
    {
        int outer_id = omp_get_thread_num();
        printf("outer thread %d\n",outer_id);
    #pragma omp parallel
    {
        int inner_id = omp_get_thread_num();
        printf("Inner thread %d outer thread %d\n",inner_id,outer_id);
    }
        }
    return 0;
}
```

Output:

Case 1:

```
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ gcc 15.c -fopenmp -o 15
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ ./15
outer thread 0
outer thread 1
Inner thread 0 outer thread 0
Inner thread 1 outer thread 0
Inner thread 0 outer thread 1
Inner thread 1 outer thread 1
```

Case 2:

```
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ gcc 15.c -fopenmp -o 15
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ ./15
outer thread 0
outer thread 2
```

```
outer thread 1
outer thread 3
Inner thread 1 outer thread 0
Inner thread 0 outer thread 0
Inner thread 3 outer thread 0
Inner thread 2 outer thread 0
Inner thread 1 outer thread 3
Inner thread 2 outer thread 3
Inner thread 0 outer thread 3
Inner thread 3 outer thread 3
Inner thread 2 outer thread 2
Inner thread 3 outer thread 2
Inner thread 1 outer thread 2
Inner thread 0 outer thread 2
Inner thread 2 outer thread 1
Inner thread 0 outer thread 1
Inner thread 3 outer thread 1
Inner thread 1 outer thread 1
```

Q26. Write an OpenMP program using #pragma omp task to compute Fibonacci numbers recursively. Create tasks for recursive calls.

Source Code:

```
#include<stdio.h>
#include<omp.h>
int fib(int n){
    if(n<=1)
        return n;
    int x,y;
#pragma omp task shared(x)
    x = fib(n-1);
#pragma omp task shared(y)
    y = fib(n-2);
#pragma omp taskwait
    return x+y;
}
int main(){
    int n = 10;
    int result;
#pragma omp parallel
    {
#pragma omp single
        result = fib(n);
    }
    printf("The Fibonacci(%d) numbers = %d\n",n,result);
    return 0;
}
```

Output:

Case 1:

```
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ gcc 16.c -fopenmp -o 16
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ ./16
The Fibonacci(10) numbers = 55
```

Case 2:

```
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ gcc 16.c -fopenmp -o 16
sukdeb@SUKDEB:~/sukdeb/HPC/Open_MP/Assignment$ ./16
The Fibonacci(15) numbers = 610
```

CUDA Programming Assignments

Q27. Write a CUDA program where each GPU thread prints "I am thread X in block Y".

Source code:

```
%%cuda
#include<stdio.h>
#include<cuda_runtime.h>
__global__ void fun(){
          printf("i   am   thread   %d   my   global   thread   id   is   %d   in
block %d!\n",threadIdx.x,blockDim.x*blockIdx.x+threadIdx.x,blockIdx.x);
}
int main(){
    fun<<<2,5>>>();
    cudaDeviceSynchronize();
    return 0;
}
```

Output:

Case 1:

```
i am thread 0 my global thread id is 5 in block 1!
i am thread 1 my global thread id is 6 in block 1!
i am thread 2 my global thread id is 7 in block 1!
i am thread 3 my global thread id is 8 in block 1!
i am thread 4 my global thread id is 9 in block 1!
i am thread 0 my global thread id is 0 in block 0!
i am thread 1 my global thread id is 1 in block 0!
i am thread 2 my global thread id is 2 in block 0!
i am thread 3 my global thread id is 3 in block 0!
i am thread 4 my global thread id is 4 in block 0!
```

Case 2:

```
i am thread 0 my global thread id is 8 in block 2!
i am thread 1 my global thread id is 9 in block 2!
i am thread 2 my global thread id is 10 in block 2!
i am thread 3 my global thread id is 11 in block 2!
i am thread 0 my global thread id is 0 in block 0!
i am thread 1 my global thread id is 1 in block 0!
i am thread 2 my global thread id is 2 in block 0!
i am thread 3 my global thread id is 3 in block 0!
```

```
i am thread 0 my global thread id is 4 in block 1!
i am thread 1 my global thread id is 5 in block 1!
i am thread 2 my global thread id is 6 in block 1!
i am thread 3 my global thread id is 7 in block 1!
```

Q28. Write a CUDA program to add two vectors A and B of size N.

Source Code:

```
%%cuda
#include<stdio.h>
#include<stdlib.h>
#include<cuda_runtime.h>
#include<time.h>
#define N 5

__global__ void add_two_vec(int *d_arr_A,int *d_arr_B,int *d_arr_C){
    int i = blockIdx.x*blockDim.x+threadIdx.x;
    if(i < N)
    d_arr_C[i] = d_arr_A[i] + d_arr_B[i];
}

int main(){
    int *h_arr_A;
    int *h_arr_B;
    int *h_arr_C;

    int *d_arr_A;
    int *d_arr_B;
    int *d_arr_C;

    srand(time(NULL));
    h_arr_A = (int *)malloc(N*sizeof(int));
    h_arr_B = (int *)malloc(N*sizeof(int));
    h_arr_C = (int *)malloc(N*sizeof(int));

    cudaMalloc((void **)&d_arr_A,N*sizeof(int));
    cudaMalloc((void **)&d_arr_B,N*sizeof(int));
    cudaMalloc((void **)&d_arr_C,N*sizeof(int));

    printf("The Vetcor A is:\n");
    for(int i = 0;i<N;i++){
        h_arr_A[i] = rand()%100;
        printf("%d ",h_arr_A[i]);
    }
    printf("\nThe Vetcor B is:\n");
    for(int i = 0;i<N;i++){
```

```
        h_arr_B[i] = rand()%100;
        printf("%d ",h_arr_B[i]);
    }

    cudaMemcpy(d_arr_A,h_arr_A,N*sizeof(int),cudaMemcpyHostToDevice);
    cudaMemcpy(d_arr_B,h_arr_B,N*sizeof(int),cudaMemcpyHostToDevice);

    add_two_vec<<<1,5>>>(d_arr_A,d_arr_B,d_arr_C);
    cudaDeviceSynchronize();

    cudaMemcpy(h_arr_C,d_arr_C,N*sizeof(int),cudaMemcpyDeviceToHost);

    printf("\nThe resultant Vector C is:\n");
    for(int i = 0;i<N;i++){
        printf("%d ",h_arr_C[i]);
    }

    cudaFree(d_arr_A);
    cudaFree(d_arr_B);
    cudaFree(d_arr_C);

    free(h_arr_A);
    free(h_arr_B);
    free(h_arr_C);
    return 0;
}
```

Output:

Case 1:

The Vetcor A is:
75 91 33 1 90
The Vetcor B is:
53 74 70 7 15
The resultant Vector C is:
128 165 103 8 105

Case 2:

The Vetcor A is:
17 28 98 19 28

The Vetcor B is:

77 16 45 48 57

The resultant Vector C is:

94 44 143 67 85

Q29. Write a CUDA program where each thread multiplies one vector element by a constant scalar.

Source code:

```
%%cuda
#include<stdio.h>
#include<stdlib.h>
#include<cuda_runtime.h>
#include<time.h>
#define N 10

__global__ void vector_mul(int *arr,int con){
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if(i<N)
    arr[i] = arr[i] * con;
}

int main(){
    int size = N*sizeof(int);
    srand(time(NULL));

    int *h_arr;
    h_arr = (int *)malloc(size);
    int *d_arr;
    cudaMalloc((void **)&d_arr,size);
    int h_con; // using normal variable

    printf("the vector is:\n");
    for(int i = 0;i<N;i++){
        h_arr[i] = rand()%20;
        printf("%d ",h_arr[i]);
    }
    h_con = rand()%20;
    printf("\nthe constant is: %d",h_con);

    cudaMemcpy(d_arr,h_arr,size,cudaMemcpyHostToDevice);

    vector_mul<<<2,5>>>(d_arr,h_con);
    cudaDeviceSynchronize();
    cudaMemcpy(h_arr,d_arr,size,cudaMemcpyDeviceToHost);
```

```
    printf("\nthe vector is:\n");
    for(int i = 0;i<N;i++){
        printf("%d ",h_arr[i]);
    }
    cudaFree(d_arr);
    free(h_arr);

    return 0;
}
```

Output:

Case1:

The vector is:
13 14 18 4 5 13 4 5 12 2
the constant is: 10
the vector is:
130 140 180 40 50 130 40 50 120 20

Case 2:

The vector is:
15 6 10 19 16 4 19 10 0 12
the constant is: 6
the vector is:
90 36 60 114 96 24 114 60 0 72

Q30. Write a CUDA kernel where each thread checks whether its assigned number is even or odd and store results in another array.

Source code:

```
%%cuda
#include<stdio.h>
#include<stdlib.h>
#include<cuda_runtime.h>
#include<time.h>
#define N 10

__global__ void evv_odd(int * arr_EvOd,int * res){
    int i = blockDim.x * blockIdx.x + threadIdx.x;
    if (i<N)
    res[i] = arr_EvOd[i]%2;
}
int main(){
    int size = N*sizeof(int);
    int *h_arr;
    int *d_arr;
    int *h_res;
    int *d_res;
    srand(time(NULL));

    h_arr = (int *)malloc(size);
    h_res = (int *)malloc(size);

    cudaMalloc((void **)&d_arr,size);
    cudaMalloc((void **)&d_res,size);

    printf("the array is:\n");
    for(int i = 0;i<N;i++){
        h_arr[i] = rand()%100;
        printf("%d ",h_arr[i]);
    }
    cudaMemcpy(d_arr,h_arr,size,cudaMemcpyHostToDevice);

    evv_odd<<<2,5>>>(d_arr,d_res);
    cudaDeviceSynchronize();

    cudaMemcpy(h_res,d_res,size,cudaMemcpyDeviceToHost);
```

```
    printf("\nThe Result array: \n");
    for(int i = 0;i<N;i++){
        printf("%d is ",h_arr[i]);
        if(h_res[i] == 0){
            printf("even");
        }
        else{
            printf("odd");
        }
        printf("\n");
    }
    cudaFree(d_arr);
    cudaFree(d_res);
    free(h_arr);
    free(h_res);

    return 0;
}
```

Output:

Case1:

The array is:
86 93 12 86 4 85 54 24 61 65
The Result array:
86 is even
93 is odd
12 is even
86 is even
4 is even
85 is odd
54 is even
24 is even
61 is odd
65 is odd

Case 2:

The array is:
36 26 1 45 71 18 63 15 97 90
The Result array:
36 is even

26 is even
1 is odd
45 is odd
71 is odd
18 is even
63 is odd
15 is odd
97 is odd
90 is even

Q31. Write a CUDA program to reverse an array in parallel.

Source code:

```
%%cuda
#include<stdio.h>
#include<stdlib.h>
#include<time.h>
#include<cuda_runtime.h>
#define N 10


__global__ void reverse(int * arr,int * rev){
    int i = blockDim.x * blockIdx.x + threadIdx.x;
    if (i < N)
    rev[i] = arr[N-i-1];
}

int main(){
    int *h_arr,*h_rev;
    int *d_arr,*d_rev;
    int size = N*sizeof(int);
    srand(time(NULL));

    h_arr = (int *)malloc(size);
    h_rev = (int *)malloc(size);
    cudaMalloc((void **)&d_arr,size);
    cudaMalloc((void **)&d_rev,size);

    printf("The Original array is:\n");
    for (int i = 0;i<N;i++){
        h_arr[i] = rand()%100;
        printf("%d ",h_arr[i]);
    }

    cudaMemcpy(d_arr,h_arr,size,cudaMemcpyHostToDevice);

    reverse<<<2,N/2>>>(d_arr,d_rev);

    cudaMemcpy(h_rev,d_rev,size,cudaMemcpyDeviceToHost);
    cudaDeviceSynchronize();
```

```
    printf("\nAfter reversing the original array is:\n");
    for (int i = 0;i<N;i++){
        printf("%d ",h_rev[i]);
    }

    cudaFree(d_arr);
    cudaFree(d_rev);
    free(h_arr);
    free(h_rev);

    return 0;
}
```

Output:

Case1:

The Original array is:
73 90 74 21 56 45 49 64 36 18
After reversing the original array is:
18 36 64 49 45 56 21 74 90 73

Case 2:

The Original array is:
61 81 22 34 88 50 15 98 89 87
After reversing the original array is:
87 89 98 15 50 88 34 22 81 61

Q32. Write a CUDA program to search a key element in an array.

Source code:

```
%%cuda
#include<stdio.h>
#include<stdlib.h>
#include<cuda_runtime.h>
#include<time.h>
#define N 10


__global__ void search(int *arr,int key,int *is_found){
    int i = blockDim.x*blockIdx.x+threadIdx.x;
    if(i<N){
        if (arr[i] == key){
            atomicExch(is_found,1);
        }
    }
}

int main(){
    int size = N*sizeof(int);
    int *h_arr,*h_is_found,key;
    int *d_arr,*d_is_found;
    srand(time(NULL));

    h_arr = (int *)malloc(size);
    h_is_found = (int *)malloc(sizeof(int));

    cudaMalloc((void **)&d_arr,size);
    cudaMalloc((void **)&d_is_found,sizeof(int));
    printf("The Array is:\n");
    for(int i = 0;i<N;i++){
        h_arr[i] = rand()%15;
        // h_arr[i] = i+10;
        printf("%d ",h_arr[i]);
    }
    printf("\n");
    key = 10;
    printf("The key is: %d\n",key);
    *h_is_found = 0;
```

```
    cudaMemcpy(d_arr,h_arr,size,cudaMemcpyHostToDevice);
    cudaMemcpy(d_is_found,h_is_found,sizeof(int),cudaMemcpyHostToDevice);

    search<<<3,5>>>(d_arr,key,d_is_found);
    cudaDeviceSynchronize();

    cudaMemcpy(h_is_found,d_is_found,sizeof(int),cudaMemcpyDeviceToHost);
    if(*h_is_found){
        printf("key is found");
    }
    else{
        printf("key is not found");
    }

    cudaFree(d_arr);
    cudaFree(d_is_found);
    free(h_arr);
    free(h_is_found);

    return 0;
}
```

Output:

Case1:

The Array is:
2 5 1 8 4 11 12 14 5 4
The key is: 10
key is not found

Case 2:

The Array is:
14 10 10 11 1 1 5 12 7 7
The key is: 5
key is found

Q33. Write a CUDA program where each thread processes part of the array and find the maximum element using parallel reduction.

Source code:

```
%%cuda
#include<stdio.h>
#include<stdlib.h>
#include<time.h>

#define N 8   // better power of 2

__global__ void maximum(int *arr){
    int tid = threadIdx.x;

    for(int stride = blockDim.x/2; stride > 0; stride /= 2){
        if(tid < stride){
            if(arr[tid] < arr[tid + stride]){
                arr[tid] = arr[tid + stride];
            }
        }
        __syncthreads();
    }
}

int main(){
    int *h_arr;
    int *d_arr;
    int size = N * sizeof(int);

    srand(time(NULL));

    h_arr = (int*)malloc(size);
    cudaMalloc((void**)&d_arr, size);

    printf("The Original array:\n");
    for(int i = 0; i < N; i++){
        h_arr[i] = rand() % 50;
        printf("%d ", h_arr[i]);
    }
    printf("\n");
```

```
    cudaMemcpy(d_arr, h_arr, size, cudaMemcpyHostToDevice);

    maximum<<<1, N>>>(d_arr);

    cudaMemcpy(h_arr, d_arr, size, cudaMemcpyDeviceToHost);

    printf("The Maximum element in this array: %d\n", h_arr[0]);

    cudaFree(d_arr);
    free(h_arr);

    return 0;
}
```

Output:

Case1:

The Original array:
27 42 30 26 4 41 28 5
Maximum element in this array: 42

Case 2:

The Original array:
12 37 36 24 17 33 2 15
The Maximum element in this array: 37

Q34. Write a CUDA program to compute dot product of two vectors.
Source code:

```
%%cuda
#include<stdio.h>
#include<stdlib.h>
#include<cuda_runtime.h>
#include<time.h>
#define N 3

__global__ void dot_prod(int *arr1,int *arr2,int *sum){
    int i = blockDim.x*blockIdx.x+threadIdx.x;
    if(i<N)
    atomicAdd(sum,arr1[i]*arr2[i]);
}

int main(){
    int size = N*sizeof(int);
    int *h_arr_A,*h_arr_B,*h_sum;
    int *d_arr_A,*d_arr_B,*d_sum;
    srand(time(NULL));

    h_arr_A = (int *)malloc(size);
    h_arr_B = (int *)malloc(size);
    h_sum = (int *)malloc(sizeof(int));
    *h_sum = 0;

    cudaMalloc((void **)&d_arr_A,size);
    cudaMalloc((void **)&d_arr_B,size);
    cudaMalloc((void **)&d_sum,sizeof(int));

    for(int i = 0;i<N;i++){
            h_arr_A[i] = rand()%100;
            h_arr_B[i] = rand()%100;
    }

    printf("The Vector A:\n");
    for(int i = 0;i<N;i++){
            printf("%d ",h_arr_A[i]);
    }
        printf("\n");
    printf("The Vector B:\n");
```

```
    for(int i = 0;i<N;i++){
            printf("%d ",h_arr_B[i]);
    }

    cudaMemcpy(d_arr_A,h_arr_A,size,cudaMemcpyHostToDevice);
    cudaMemcpy(d_arr_B,h_arr_B,size,cudaMemcpyHostToDevice);
    cudaMemcpy(d_sum,h_sum,size,cudaMemcpyHostToDevice);

    dot_prod<<<1,N>>>(d_arr_A,d_arr_B,d_sum);
    cudaDeviceSynchronize();

    cudaMemcpy(h_sum,d_sum,sizeof(int),cudaMemcpyDeviceToHost);

    printf("\nThe result of dot product of two vectors is: %d",*h_sum);

    cudaFree(d_arr_A);
    cudaFree(d_arr_B);
    cudaFree(d_sum);
    free(h_arr_A);
    free(h_arr_B);
    free(h_sum);

    return 0;

}
```

Output:

Case1:

The Vector A:
51 62 81
The Vector B:
19 24 3
The result of dot product of two vectors is: 2700

Case 2:

The Vector A:
1 22 88
The Vector B:
37 74 80
The result of dot product of two vectors is: 8705

Q35. Given an array of numbers from 0–50, compute the frequency of each number using CUDA threads.
Source code:

```
%%cuda
#include<stdio.h>
#include<stdlib.h>
#include<cuda_runtime.h>
#include<time.h>
#define N 50
#define range 51

__global__ void freq(int *arr,int *freq_arr){
    int i = blockDim.x*blockIdx.x+threadIdx.x;
    if (i<N){
        int temp = arr[i];
        atomicAdd(&freq_arr[temp],1);
    }
}

int main(){
    int size = N*sizeof(int);
    int h_arr[N],h_freq_arr[range]={0};
    int *d_arr,*d_freq_arr;
    srand(time(NULL));

    cudaMalloc((void **)&d_arr,size);
    cudaMalloc((void **)&d_freq_arr,range*sizeof(int));

    printf("The vector is:\n");
    for(int i = 0;i<N;i++){
        h_arr[i]=rand()%51;
        printf("%d ",h_arr[i]);
    }

    cudaMemcpy(d_arr,h_arr,size,cudaMemcpyHostToDevice);
    cudaMemcpy(d_freq_arr,h_freq_arr,range*sizeof(int),cudaMemcpyHostToDevice);

    freq<<<1,N>>>(d_arr,d_freq_arr);
    cudaDeviceSynchronize();

    cudaMemcpy(h_freq_arr,d_freq_arr,range*sizeof(int),cudaMemcpyDeviceToHost);
```

```
    printf("\n");
    for(int i = 0;i<range;i++){
        if(h_freq_arr[i]!=0)
        printf("%d occured %d times\n",i,h_freq_arr[i]);
    }

    cudaFree(d_arr);
    cudaFree(d_freq_arr);
}
```

Output:

Case1:

The vector is:
28 11 19 20 50 7 44 30 44 8 5 32 43 22 11 14 14 22 15 13 0 10 10 33 42 45 11 0 32 31 36 10 42 4 4 15 36 22 20 30 30 25 36 22 21 21 10 9 43 25
0 occured 2 times
4 occured 2 times
5 occured 1 times
7 occured 1 times
8 occured 1 times
9 occured 1 times
10 occured 4 times
11 occured 3 times
13 occured 1 times
14 occured 2 times
15 occured 2 times
19 occured 1 times
20 occured 2 times
21 occured 2 times
22 occured 4 times
25 occured 2 times
28 occured 1 times
30 occured 3 times
31 occured 1 times
32 occured 2 times
33 occured 1 times
36 occured 3 times
42 occured 2 times
43 occured 2 times
44 occured 2 times

45 occured 1 times
50 occured 1 times

Case 2:

The vector is:
26 5 40 2 41 28 39 41 33 43 39 37 46 20 27 27 20 40 37 41 47 35 20 33 46 19 6 7 4 22 4 30 2 44 6 43 46 45 7 3 12 47 40 7 16 41 8 37 30 45
2 occured 2 times
3 occured 1 times
4 occured 2 times
5 occured 1 times
6 occured 2 times
7 occured 3 times
8 occured 1 times
12 occured 1 times
16 occured 1 times
19 occured 1 times
20 occured 3 times
22 occured 1 times
26 occured 1 times
27 occured 2 times
28 occured 1 times
30 occured 2 times
33 occured 2 times
35 occured 1 times
37 occured 3 times
39 occured 2 times
40 occured 3 times
41 occured 4 times
43 occured 2 times
44 occured 1 times
45 occured 2 times
46 occured 3 times
47 occured 2 times

Q36. Write a CUDA program to add two matrices.

Source code:

```
%%cuda
#include<stdio.h>
#include<stdlib.h>
#include<cuda_runtime.h>
#include<time.h>
#define N 3

__global__ void add(int *arr1,int *arr2,int *arr3){
    int i = blockDim.x*blockIdx.x+threadIdx.x;
    if(i<N*N)
    arr3[i] = arr2[i] + arr1[i];
}

int main(){
    int size = N*N*sizeof(int);
    int *h_arr_A,*h_arr_B,*h_arr_C;
    int *d_arr_A,*d_arr_B,*d_arr_C;
    srand(time(NULL));

    h_arr_A = (int *)malloc(size);
    h_arr_B = (int *)malloc(size);
    h_arr_C = (int *)malloc(size);

    cudaMalloc((void **)&d_arr_A,size);
    cudaMalloc((void **)&d_arr_B,size);
    cudaMalloc((void **)&d_arr_C,size);

    for(int i = 0;i<N;i++){
        for(int j = 0;j<N;j++){
            h_arr_A[i*N+j] = rand()%100;
            h_arr_B[i*N+j] = rand()%100;
        }
    }

    printf("The Matrix A:\n");
    for(int i = 0;i<N;i++){
        for(int j = 0;j<N;j++){
            printf("%d ",h_arr_A[i*N+j]);
```

```
        }
        printf("\n");
    }
        printf("\n");
    printf("The Matrix B:\n");
    for(int i = 0;i<N;i++){
        for(int j = 0;j<N;j++){
            printf("%d ",h_arr_B[i*N+j]);
        }
        printf("\n");
    }

    cudaMemcpy(d_arr_A,h_arr_A,size,cudaMemcpyHostToDevice);
    cudaMemcpy(d_arr_B,h_arr_B,size,cudaMemcpyHostToDevice);

    add<<<N,N>>>(d_arr_A,d_arr_B,d_arr_C);
    cudaDeviceSynchronize();

    cudaMemcpy(h_arr_C,d_arr_C,size,cudaMemcpyDeviceToHost);

    printf("\nThe resultant Matrix C:\n");
    for(int i = 0;i<N;i++){
        for(int j = 0;j<N;j++){
            printf("%d ",h_arr_C[i*N+j]);
        }
        printf("\n");
    }

    cudaFree(d_arr_A);
    cudaFree(d_arr_B);
    cudaFree(d_arr_C);
    free(h_arr_A);
    free(h_arr_B);
    free(h_arr_C);

    return 0;

}
```

Output:

Case1:

```
The Matrix A:
66 27 60
48 2 94
23 9 55

The Matrix B:
14 63 3
59 50 10
1 73 76

The resultant Matrix C:
80 90 63
107 52 104
24 82 131
```

Case 2:

```
The Matrix A:
66 61 30
74 26 36
82 24 43

The Matrix B:
10 11 27
70 72 31
81 27 26

The resultant Matrix C:
76 72 57
144 98 67
163 51 69
```

Q37. Write a CUDA program to multiply two matrices.

Source code:

```
%%cuda
#include<stdio.h>
#include<stdlib.h>
#include<time.h>
#define row_1 3
#define col_1 5
#define row_2 5
#define col_2 3

__global__ void mult(int * arr1,int * arr2, int * arr3,int i,int k,int j){
    int c = blockDim.x*blockIdx.x+threadIdx.x;
    int r = blockDim.y*blockIdx.y+threadIdx.y;
    if(r < i && c < j){
        int sum = 0;
        for(int a = 0;a<k;a++){
            sum += arr1[r*k+a] * arr2[a*j+c];
        }
        arr3[r*j+c] = sum;
    }
}

int main(){
    srand(time(NULL));
    if(col_1 == row_2){
      int arr_1[row_1][col_1];
      int size_1 = row_1 * col_1 * sizeof(int);
      int size_2 = row_2 * col_2 * sizeof(int);
      int size_3 = row_1 * col_2 * sizeof(int);
      printf("\nthe matrix A:\n");
      for(int i = 0;i<row_1;i++){
          for(int j = 0;j<col_1;j++){
              arr_1[i][j] = rand()%20;
              printf("%d ",arr_1[i][j]);
          }
          printf("\n");
      }

      int arr_2[row_2][col_2];
```

```
        printf("\nThe matrix B:\n");
        for(int i = 0;i<row_2;i++){
            for(int j = 0;j<col_2;j++){
                arr_2[i][j] = rand()%20;
                printf("%d ",arr_2[i][j]);
            }
            printf("\n");
        }
        int arr_3[row_1][col_2]={0};

        int *d_arr1,*d_arr2,*d_arr3;

        cudaMalloc((void **)&d_arr1,size_1);
        cudaMalloc((void **)&d_arr2,size_2);
        cudaMalloc((void **)&d_arr3,size_3);

        cudaMemcpy(d_arr1,arr_1,size_1,cudaMemcpyHostToDevice);
        cudaMemcpy(d_arr2,arr_2,size_2,cudaMemcpyHostToDevice);
        cudaMemcpy(d_arr3,arr_3,size_3,cudaMemcpyHostToDevice);

        dim3 threads(16,16);
        dim3 blocks((col_2+15)/16 , (row_1+15)/16);

        mult<<<blocks,threads>>>(d_arr1,d_arr2,d_arr3,row_1,col_1,col_2);
        cudaDeviceSynchronize();

        cudaMemcpy(arr_3,d_arr3,size_3,cudaMemcpyDeviceToHost);

        printf("\nthe result matrix C:\n");
        for(int i = 0;i<row_1;i++){
            for(int j = 0;j<col_2;j++){
                printf("%d ",arr_3[i][j]);
            }
            printf("\n");
        }
    }
    else{
        printf("dimention of the matrix is not multipliable\n");
    }

}
```

Output:

Case1:

```
The matrix A:
3 17 17 16 12
9 13 15 7 16
8 6 12 17 18

The matrix B:
6 5 14
16 14 16
8 0 1
18 1 2
8 5 11

The result matrix C:
810 329 495
636 314 539
690 231 452
```

Case 2:

```
The matrix A:
15 10 16 5 3
14 9 5 4 17
12 6 0 16 11

The matrix B:
1 10 5
19 16 15
0 17 3
17 15 2
3 5 8

The result matrix C:
299 672 307
304 514 364
431 511 270
```

Q38. Write a CUDA program to compute matrix transpose.

Source code:

```
%%cuda
#include<stdio.h>
#include<stdlib.h>
#include<time.h>
#define row_1 3
#define col_1 6

__global__ void mult(int * arr1,int * arr2,int i,int j){
    int c = blockDim.x*blockIdx.x+threadIdx.x;
    int r = blockDim.y*blockIdx.y+threadIdx.y;
    if(r < i && c < j){
        arr2[c * i + r] = arr1[r * j + c];
    }
}

int main(){
    srand(time(NULL));
      int arr_1[row_1][col_1];
      int arr_2[col_1][row_1];
      printf("\nThe matrix A\n");
      for(int i = 0;i<row_1;i++){
          for(int j = 0;j<col_1;j++){
              arr_1[i][j] = rand()%100;
              printf("%d ",arr_1[i][j]);
          }
          printf("\n");
      }

      int *d_arr1,*d_arr2;

      int size_1 = row_1 * col_1 * sizeof(int);
      cudaMalloc((void **)&d_arr1,size_1);
      cudaMalloc((void **)&d_arr2,size_1);

      cudaMemcpy(d_arr1,arr_1,size_1,cudaMemcpyHostToDevice);

      dim3 threads(16,16);
      dim3 blocks((col_1+15)/16 , (row_1+15)/16);
```

```
    mult<<<blocks,threads>>>(d_arr1,d_arr2,row_1,col_1);
    cudaDeviceSynchronize();

    cudaMemcpy(arr_2,d_arr2,size_1,cudaMemcpyDeviceToHost);

    printf("\nThis is transposed matrix\n");
    for(int i = 0;i<col_1;i++){
        for(int j = 0;j<row_1;j++){
            printf("%d ",arr_2[i][j]);
        }
        printf("\n");
    }

}
```

Output:

Case1:

The matrix A
94 29 55 76 12 16
86 60 8 35 57 19
31 49 47 59 51 65

This is transposed matrix
94 86 31
29 60 49
55 8 47
76 35 59
12 57 51
16 19 65

Case 2:

The matrix A
31 76 24 36 51 89
29 1 82 95 50 80
66 8 7 60 82 43

This is transposed matrix

31 29 66

76 1 8

24 82 7

36 95 60

51 50 82

89 80 43

Q39. Consider a grayscale image represented as a 2D array. Write a CUDA program where each thread increases pixel intensity by a constant value. Ensure pixel values remain within valid range.

Source code:

```
%%cuda
#include<stdio.h>
#include<stdlib.h>
#include<time.h>
#define row_1 5
#define col_1 5
#define cons 10

__global__ void mult(int * arr1,int * arr2,int i,int j){
    int c = blockDim.x*blockIdx.x+threadIdx.x;
    int r = blockDim.y*blockIdx.y+threadIdx.y;
    if(r < i && c < j){
        arr2[r*j+c] = min(arr1[r*j+c] + cons,255) ;
    }
}

int main(){
    srand(time(NULL));
      int arr_1[row_1][col_1];
      int arr_2[row_1][col_1];
      printf("\nthis is gray scale image\n");
      for(int i = 0;i<row_1;i++){
          for(int j = 0;j<col_1;j++){
              arr_1[i][j] = rand()%256;
              printf("%3d ",arr_1[i][j]);
          }
          printf("\n");
      }

      int *d_arr1,*d_arr2;

      int size_1 = row_1 * col_1 * sizeof(int);
      cudaMalloc((void **)&d_arr1,size_1);
      cudaMalloc((void **)&d_arr2,size_1);

      cudaMemcpy(d_arr1,arr_1,size_1,cudaMemcpyHostToDevice);
```

```
    dim3 threads(16,16);
    dim3 blocks((col_1+15)/16 , (row_1+15)/16);

    mult<<<blocks,threads>>>(d_arr1,d_arr2,row_1,col_1);
    cudaDeviceSynchronize();

    cudaMemcpy(arr_2,d_arr2,size_1,cudaMemcpyDeviceToHost);

    printf("\nThis is after increased intensity of each pixel\n");
    for(int i = 0;i<row_1;i++){
        for(int j = 0;j<col_1;j++){
            printf("%3d ",arr_2[i][j]);
        }
        printf("\n");
    }

}
```

Output:

```
This is gray scale image
  1 215 126 194  30
163 233 247 147  65
  8 115 167 157 160
183  75  47 255 171
 15 241  59 184  37

This is after increased intensity of each pixel
 11 225 136 204  40
173 243 255 157  75
 18 125 177 167 170
193  85  57 255 181
 25 251  69 194  47
```

Q40. Write a CUDA program where each thread processes a pixel. If the pixel value is greater than a threshold, then set it to 255, otherwise, set it to 0.

Source code:

```
%%cuda

#include<stdio.h>

#include<stdlib.h>

#include<time.h>

#define row_1 10

#define col_1 10

#define cons 20


__global__ void mult(int * arr1,int * arr2,int i,int j){

    int c = blockDim.x*blockIdx.x+threadIdx.x;

    int r = blockDim.y*blockIdx.y+threadIdx.y;

    if(r < i && c < j){

        if(arr1[r*j+c] >= 255)

            arr2[r*j+c] = 255;

        else

            arr2[r*j+c] = 0;

    }

}


int main(){

    srand(time(NULL));

      int arr_1[row_1][col_1];

      int arr_2[row_1][col_1];

      printf("\nThis is gray scale image\n");

      for(int i = 0;i<row_1;i++){
```

```
    for(int j = 0;j<col_1;j++){
        arr_1[i][j] = rand()%512;
        printf("%3d ",arr_1[i][j]);
    }
    printf("\n");
}

int *d_arr1,*d_arr2;

int size_1 = row_1 * col_1 * sizeof(int);
cudaMalloc((void **)&d_arr1,size_1);
cudaMalloc((void **)&d_arr2,size_1);

cudaMemcpy(d_arr1,arr_1,size_1,cudaMemcpyHostToDevice);

dim3 threads(16,16);
dim3 blocks((col_1+15)/16 , (row_1+15)/16);

mult<<<blocks,threads>>>(d_arr1,d_arr2,row_1,col_1);
cudaDeviceSynchronize();

cudaMemcpy(arr_2,d_arr2,size_1,cudaMemcpyDeviceToHost);

printf("\nThis is after normalized each pixel\n");
for(int i = 0;i<row_1;i++){
    for(int j = 0;j<col_1;j++){
        printf("%3d ",arr_2[i][j]);
    }
```

```
        printf("\n");
    }


}
```

Output:

Case 1:

```
This is gray scale image
490 307   2 229  41
 32  35 480  17 500
299 136 224 142 103
 74 201 146 497  36
119 246  71 390 372

This is after normalized each pixel
255 255   0   0   0
  0   0 255   0 255
255   0   0   0   0
  0   0   0 255   0
  0   0   0 255 255
```

Case2:

```
This is gray scale image
140 112 496  90 464 489 438 118 344 393
158 246 173  48 113  20 405 239 456 319
 20 452 400 124 264 497 479 324 374 472
254   2  72 238  92  24 215  18 142  47
411 300 293  72 349 407  93 242 134  37
 50 154 489 450 278 241 436 245  53 298
205 307 300 278  33 392 302 248 410 445
295 310 233  77 382  70 484 475 313 106
  0 363 260 489 301  26 218 225 271 272
 11 476  67 311 242 101 191  33 349  90
```

This is after normalized each pixel

0	0	255	0	255	255	255	0	255
255	255	255	0	255	255	0	0	0
0	0	255	255	255	0	255	0	0
255	255	0	0	255	0	255	255	255
0	255	255	255	255	0	0	0	255
0	255	0	255	0	0	0	0	255

Q41. Write a CUDA program using 3D thread blocks to process volumetric data where each thread processes one voxel and applies a simple operation.

Source code:

```
%%cuda
#include<stdio.h>
#include<stdlib.h>
#include<time.h>
#define row_1 3
#define col_1 3
#define brd_1 3
__global__ void mult(int *A,int *B,int rows,int cols,int depth)
{
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;
    int z = blockIdx.z * blockDim.z + threadIdx.z;

    if(x < cols && y < rows && z < depth)
    {
        int index = z*(rows*cols) + y*cols + x;

        B[index] = A[index] * 2;
    }
}
int main(){
    srand(time(NULL));
      int arr_1[row_1][col_1][brd_1];
      int arr_2[col_1][row_1][brd_1];
      printf("\nThis is 3D cube:\n");
      for(int i = 0;i<row_1;i++){
          for(int j = 0;j<col_1;j++){
              for(int k = 0;k<brd_1;k++){

              arr_1[i][j][k] = rand()%10;
              printf("%d ",arr_1[i][j][k]);
              }
          printf("\n");
          }
          printf("\n");
      }
```

```
      int *d_arr1,*d_arr2;
      int size_1 = row_1 * col_1 * brd_1 * sizeof(int);
      cudaMalloc((void **)&d_arr1,size_1);
      cudaMalloc((void **)&d_arr2,size_1);
      cudaMemcpy(d_arr1,arr_1,size_1,cudaMemcpyHostToDevice);
    dim3 threads(8,8,8);
dim3 blocks(
    (col_1+7)/8,
    (row_1+7)/8,
    (brd_1+7)/8
);
      mult<<<blocks,threads>>>(d_arr1,d_arr2,row_1,col_1,brd_1);
      cudaDeviceSynchronize();

      cudaMemcpy(arr_2,d_arr2,size_1,cudaMemcpyDeviceToHost);

      printf("\nThis is the cube after simple calculation with each voxel:\n");
      for(int i = 0;i<col_1;i++){
          for(int j = 0;j<row_1;j++){
              for(int k = 0;k<brd_1;k++){
                  printf("%d ",arr_2[i][j][k]);
              }
              printf("\n");
          }
          printf("\n");
      }

    }
```

Output:

Case1:

This is 3D cube:
8 1 6
2 1 4
9 4 8

0 2 3
8 8 8
5 6 0

2 6 6
8 4 7
9 2 5
This is the cube after simple calculation with each voxel:
16 2 12
4 2 8
18 8 16

0 4 6
16 16 16
10 12 0

4 12 12
16 8 14
18 4 10

Case 2:

This is 3D cube:
2 1 7
0 9 9
3 0 0

7 2 9
7 5 3
6 7 9

0 1 0
8 1 1
0 0 6

This is the cube after simple calculation with each voxel:
4 2 14
0 18 18
6 0 0

14 4 18
14 10 6
12 14 18

0 2 0
16 2 2
0 0 12